Lab 1 - Machine Learning (732A99) - Group A4

Anastasia Piadi (anapi501), Nazli Bilgic (nazbi056) and Max Björklund (maxbj107)

2023-11-14

Statement of Contribution

Max (maxbj107) was responsible for coding and writing analysis for assignment one.

Nazli (nazbi056) was responsible for coding and writing analysis for assignment two.

Anastasia (anapi501) was responsible for coding and writing analysis for assignment three.

Finalizing the last details of the report were done together.

Assignment 1. Handwritten digit recognition with K-nearest neighbors

The data file optdigits.csv contains information about normalized bitmaps of handwritten digits from a preprinted form from a total of 43 people. The data were first derived as 32x32 bitmaps which were then divided into nonoverlapping blocks of 4x4 and the number of on pixels are counted in each block. This has generated the resulting image of size 8x8 where each element is an integer in the range 0..16. Accordingly, each row in the data file is a sequence corresponding to 8x8 matrix, and the last element shows the actual digit from 0 to 9.

1.1

Import the data into R and divide it into training, validation and test sets (50%/25%/25%) by using the partitioning principle specified in the lecture slides.

Data imported, see appendix for code.

1.2

Use training data to fit 30-nearest neighbor classifier with function kknn() and kernel="rectangular" from package kknn and estimate :

- ***Confusion matrices for the training and test data (use table())***
- ***Misclassification errors for the training and test data***

  Comment on the quality of predictions for different digits and on the overall prediction quality.

```
## [1] "Confusion matrix for training data:"
```

```
##     pred
## true   0   1   2   3   4   5   6   7   8   9
##    0 202   0   0   0   0   0   0   0   0   0
##    1   0 179  11   0   0   0   0   1   1   3
##    2   0   1 190   0   0   0   0   1   0   0
##    3   0   0   0 185   0   1   0   1   0   1
##    4   1   3   0   0 159   0   0   7   1   4
##    5   0   0   0   1   0 171   0   1   0   8
##    6   0   2   0   0   0   0 190   0   0   0
##    7   0   3   0   0   0   0   0 178   1   0
##    8   0  10   0   2   0   0   2   0 188   2
##    9   1   3   0   5   2   0   0   3   3 183
```

```
## [1] "Confusion matrix for test data:"
```

```
##     pred
## true   0   1   2   3   4   5   6   7   8   9
##    0  77   0   0   0   1   0   0   0   0   0
##    1   0  81   2   0   0   0   0   0   0   3
##    2   0   0  98   0   0   0   0   0   3   0
##    3   0   0   0 107   0   2   0   0   1   1
##    4   0   0   0   0  94   0   2   6   2   5
##    5   0   1   1   0   0  93   2   1   0   5
##    6   0   0   0   0   0   0  90   0   0   0
##    7   0   0   0   1   0   0   0 111   0   0
##    8   0   7   0   1   0   0   0   0  70   0
##    9   0   1   1   1   0   0   0   1   0  85
```

```
## [1] "Misclassification Error for train data:"
```

```
## [1] 0.045
```

```
## [1] "Misclassification Error for test data:"
```

```
## [1] 0.053
```

The algorithm does very well in matching the data from the pixel matrices to its intended digit. The missclassification error on the training data is 4.5 %, while it rises to 5.3 % when used on the test data. This indicates that the model is a good fit as the test error is just marginally worse than the training error.

In the confusion matrix of the **training data**, we can see that the ones (1), the fours (4), the eights (8) and the nines (9) are most often miscategorized.

- The ones (1) are often misclassed as: Twos (2) or Nines (9)
- The fours (4) are often misclassed as: Sevens (7) or Nines (9)
- The eights (8) are often misclassed as: Ones (1)
- The Nines (9) are often misclassed as: Fives (5)

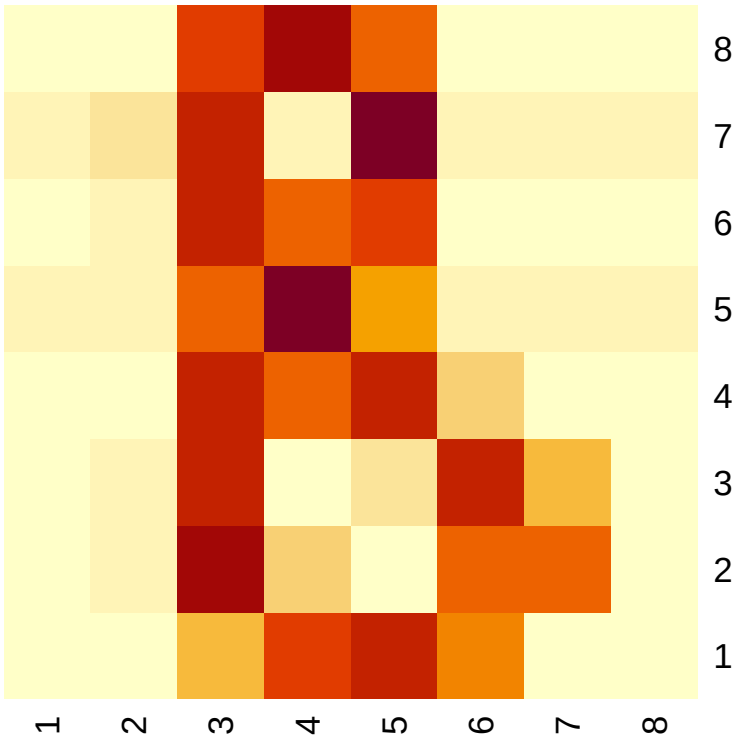
In the confusion matrix of the **test data**, we can see that the fours (4), the fives (5) and the eights (8) are most often miscategorized.

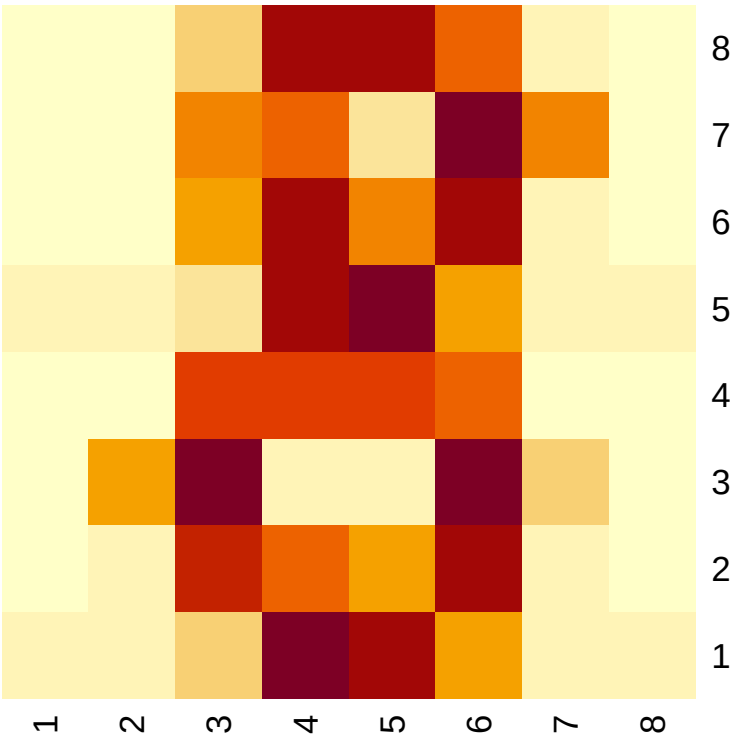
- The fours (4) are often misclassed as: Sevens (7) or Nines (9) - [**similar to training data**]
- The fives (5) are often misclassed as: Nines (9)
- The eights are often misclassed as: Ones (1)

As the model fit on both data partitions are good, the difference in digit precision between the two results compared could very well be a product of randomness.

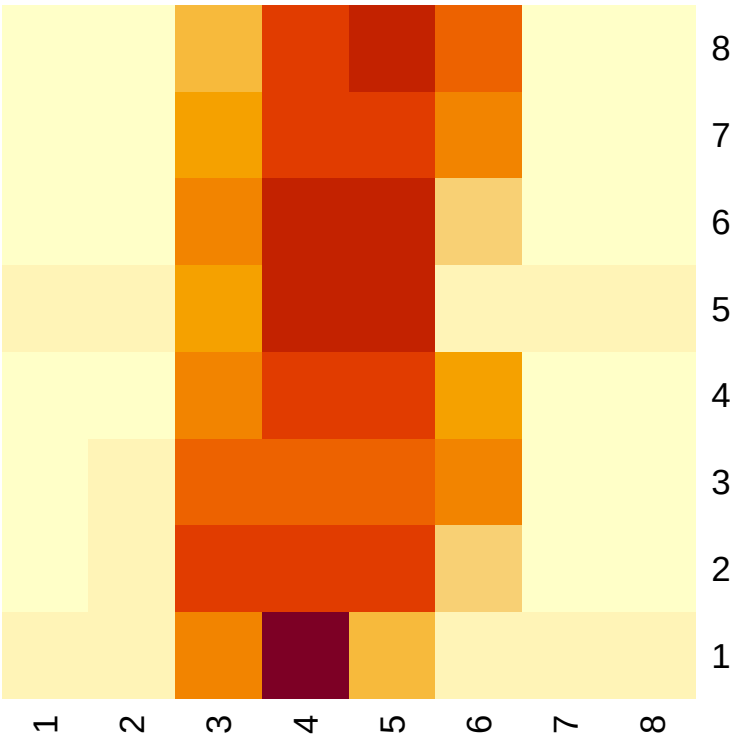
1.3

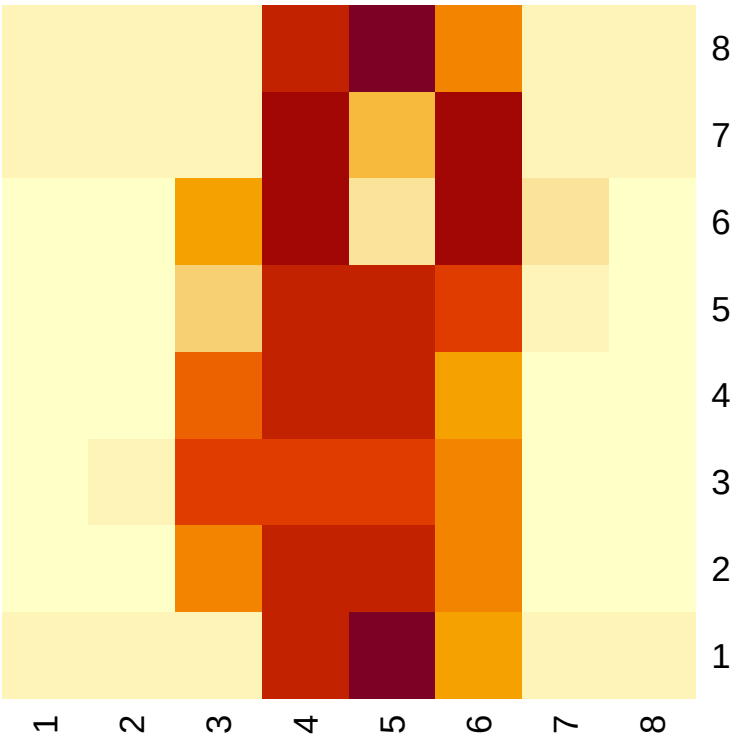
Find any 2 cases of digit "8" in the training data which were easiest to classify and 3 cases that were hardest to classify (i.e. having highest and lowest probabilities of the correct class). Reshape features for each of these cases as matrix 8x8 and visualize the corresponding digits (by using e.g. heatmap() function with parameters Colv=NA and Rowv=NA) and comment on whether these cases seem to be hard or easy to recognize visually

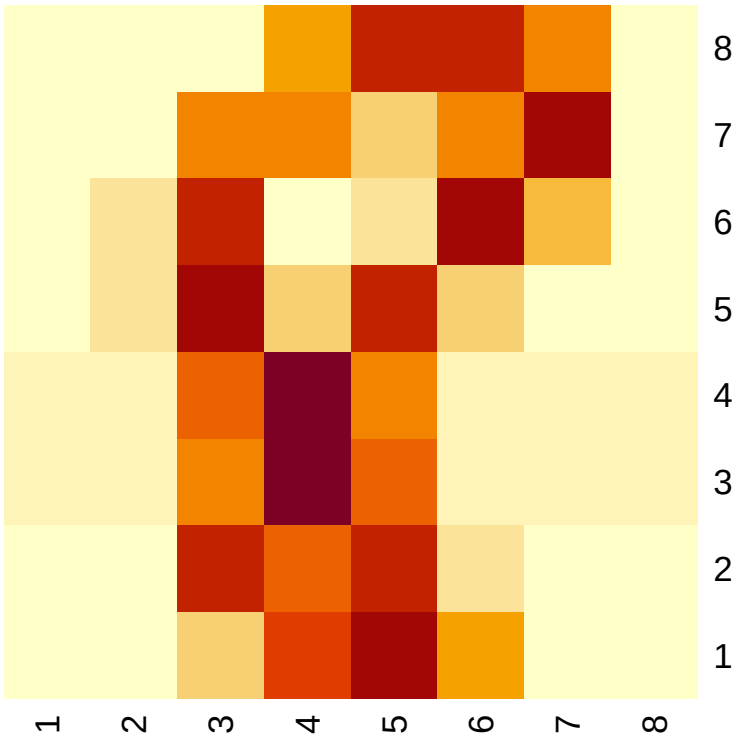




Visually, these observations clearly looks like eight, although not perfectly drawn, the rings are even split along the y-axis and contains easily distinguishable holes.


8
7
6
5
4
3
2
1
1
2
3
4
5
6
7
8


8
7
6
5
4
3
2
1
1
2
3
4
5
6
7
8



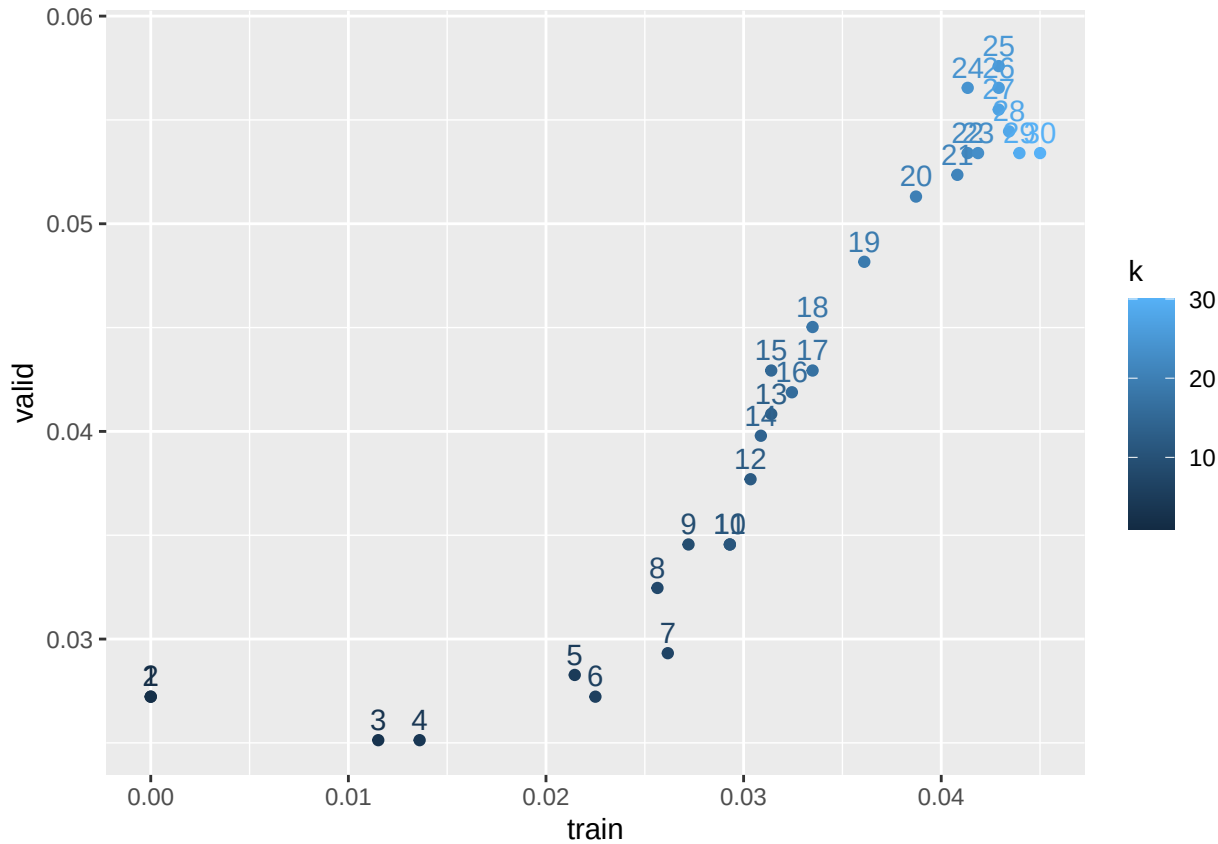
The first image does not visually look like an eight at all. It has a slim but uniform shape, as well as having no distinguishable rings.

The second image still has a shape that is to uniform, although one of the rings (the upper) can be distinguished in this one.

The third image does not have a uniform shape, but instead the digit looks very skewed. This one also has a distinguishable ring, but only the upper one.

1.4

Fit a K-nearest neighbor classifiers to the training data for different values of K =1,2, . . . , 30 and plot the dependence of the training and validation misclassification errors on the value of K (in the same plot). How does the model complexity change when K increases and how does it affect the training and validation errors? Report the optimal K according to this plot. Finally, estimate the test error for the model having the optimal K, compare it with the training and validation errors and make necessary conclusions about the model quality.



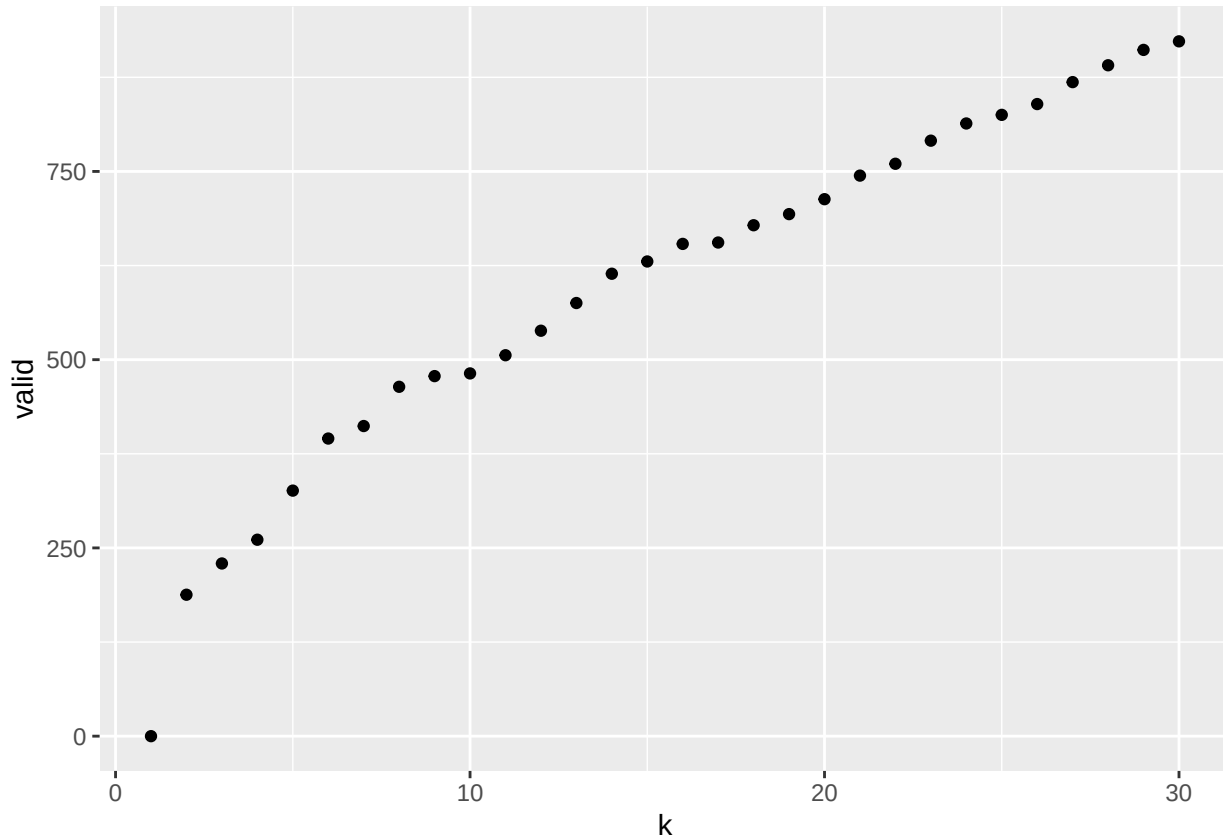
Choosing between k = 3 or k = 4 is arbitrary as the both have the lowest validation error. Choosing to go forward using k = 3.

```
##   train valid  test
## 3 0.012 0.025 0.024
```

As expected, we see the lowest error on train data, as it is fit on the same data it is trained on. The missclassification error on validation set and test set are indistinguishable. K = 3 results in low errors for all data partitions and the difference between train and valid/test is just over 1% of the observations. Neither the knn-training nor the hyperparameter-optimization seems to be over- nor underfit.

1.5

Fit K-nearest neighbor classifiers to the training data for different values of K = 1,2, . . . , 30, compute the error for the validation data as cross-entropy (when computing log of probabilities add a small constant within log, e.g. 1e-15, to avoid numerical problems) and plot the dependence of the validation error on the value ofK. What is the optimal K value here? Assuming that response has multinomial distribution, why might the cross-entropy be a more suitable choice of the error function than the misclassification error for this problem?



Using cross-entropy classification error, we see that k = 1 results in the lowest loss. Cross Entropy could be more suitable for a multinomial distributed class problem as it takes class probability into account. This means that a correctly predicted digit with 60% certainty will result in more loss than a correctly predicted digit with 90% certainty. If many digits have a similar probability for the same observation, this mean that the model is more likely to chose the wrong label.

By observing the graph, we can speculate that using a algorithm of comparing each observation to its 30-nearest-neighbours adds noise and uncertainty of which digit the observation looks most like. Comparatively, the (with good margin) best alternative of k = 1 means that the algorithm just compares each observation with the one most similar observation, and bases its sole alternative for digit label on this.

Assignment 2. Linear regression and ridge regression

2.1

Divide it into training and test data (60/40) and scale it appropriately. In the coming steps, assume that motor_UPDRS is normally distributed and is a function of the voice characteristics, and since the data are scaled, no intercept is needed in the modelling.

Data imported and scaled, see appendix for code.

2.2

Compute a linear regression model from the training data, estimate training and test MSE and comment on which variables contribute significantly to the model.

When we look at the p-values and coefficients in the model summary. Shimmer, Jitter Abs., Shimmer APQ5, Shimmer APQ11, NHR, HNR, DFA, PPE has low p-values. Variables with p-values less than significance level are considered statistically significant.

Jitter Abs., Shimmer APQ5, NHR, HNR and DFA have negative coefficient estimate. We can say that increase in these predictor variables will decrease the motor_UPDRS value. Shimmer, Shimmer.APQ11 and PPE have positive coefficient estimate. We can say that increase in these predictor variables will increase the motor_UPDRS value.

$$\text{Mean Squared Error (MSE)} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Train MSE is less than the Test MSE.

```
## Train MSE: 0.8785431
```

```
## Test MSE: 0.9354477
```

```
##
## Call:
## lm(formula = motor_UPDRS ~ . - 1, data = train_data_scaled)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.0255 -0.7363 -0.1087  0.7333  2.1960
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## Jitter...       0.186931   0.149561   1.250 0.211431
## Jitter.Abs.    -0.169609   0.040805  -4.157 3.31e-05 ***
## Jitter.RAP     -5.269544  18.834160  -0.280 0.779658
## Jitter.PPQ5    -0.074568   0.087766  -0.850 0.395592
## Jitter.DDP      5.249558  18.837525   0.279 0.780510
## Shimmer         0.592436   0.205981   2.876 0.004050 **
## Shimmer.dB.    -0.172655   0.139316  -1.239 0.215315
## Shimmer.APQ3   32.070932  77.159242   0.416 0.677694
## Shimmer.APQ5   -0.387507   0.113789  -3.405 0.000668 ***
## Shimmer.APQ11   0.305546   0.061236   4.990 6.34e-07 ***
## Shimmer.DDA   -32.387241  77.158814  -0.420 0.674695
## NHR            -0.185387   0.045567  -4.068 4.84e-05 ***
## HNR            -0.238543   0.036395  -6.554 6.41e-11 ***
## RPDE            0.004068   0.022664   0.179 0.857556
## DFA            -0.280318   0.020136 -13.921  < 2e-16 ***
## PPE             0.226467   0.032881   6.887 6.70e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9394 on 3509 degrees of freedom
## Multiple R-squared:  0.1212, Adjusted R-squared:  0.1172
## F-statistic: 30.25 on 16 and 3509 DF,  p-value: < 2.2e-16
```

2.3

Implement 4 following functions by using basic R commands only;

a) ***Loglikelihood function that for a given parameter vector theta and dispersion sigma computes the log-likelihood function log P(T/theta,sigma) for the stated model and the training data***

$$\text{Log-Likelihood} = -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log(\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - (\beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}))^2$$

Created function for log-likelihood, see appendix for code.

b) ***Ridge function that for given vector theta, scalar sigma and scalar lambda uses function from 3a and adds up a Ridge penalty lambda‖theta‖2 to the minus log-likelihood***

$$\text{Ridge penalty} = (\lambda \sum_{j=1}^{p} \theta_j^2)$$

Created function for ridge regression, see appendix for code

c) ***Ridge function that depends on scalar lambda , uses function from 3b and function optim() with method="BFGS" to find the optimal theta and sigma for the given lambda.***

Created optimization function for ridge regression, see appendix for code

d) ***DF function that for a given scalar lambda computes the degrees of freedom of the Ridge model based on the training data.***

$$\text{df}(\lambda) = \sum_{i=1}^{p} \frac{d_i^2}{d_i^2 + \lambda}$$

Creating a function for calculating degrees of freedom in ridge regression, see appendix for code

2.4

By using function RidgeOpt, compute optimal theta parameters for lambda = 1, lambda = 100 and lambda = 1000. Use the estimated parameters to predict the motor_UPDRS values for training and test data and report the training and test MSE values. Which penalty parameter is most appropriate among the selected ones? Compute and compare the degrees of freedom of these models and make appropriate conclusions.

$$\text{Mean Squared Error (MSE)} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

For the lambda; 1, 100, 1000 values, lambda=1 has the least test and train MSE values. Lambda=1000 has the highest values for the test and training MSE. For this model its better to choose lambda=1. With smaller MSE we can say that our model predictions are closer to the actual values which means better performance of the model.

For this example lambda values increase from 1 to 1000 and according to this degrees of freedom decrease.

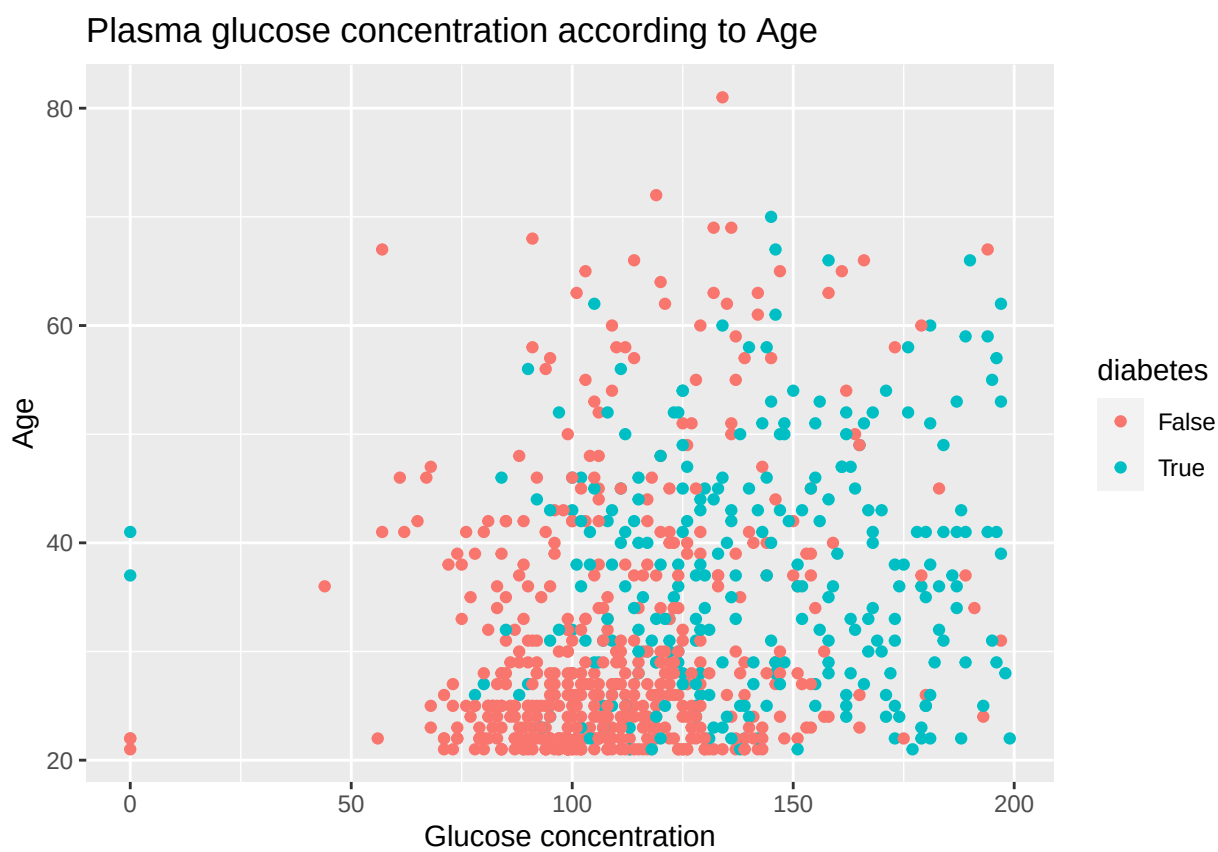
With less degrees of freedom, model can become less complex and less likely to overfit the training data.

```
## MSE for Lambda 1:
##  Train MSE: 0.8786263
##  Test MSE: 0.934958

## MSE for Lambda 100:
##  Train MSE: 0.8841219
##  Test MSE: 0.9314307

## MSE for Lambda 1000:
##  Train MSE: 0.920927
##  Test MSE: 0.9533202

## Degrees of Freedom Lambda 1: 13.86074

## Degrees of Freedom Lambda 100: 9.924887

## Degrees of Freedom  Lambda 1000: 5.643925
```

Assignment 3. Logistic regression and basis function expansion

3.1

Make a scatter plot showing a Plasma glucose concentration on Age where observations are colored by Diabetes levels. Do you think that Diabetes is easy to classify by a standard logistic regression model that uses these two variables as features? Motivate your answer.

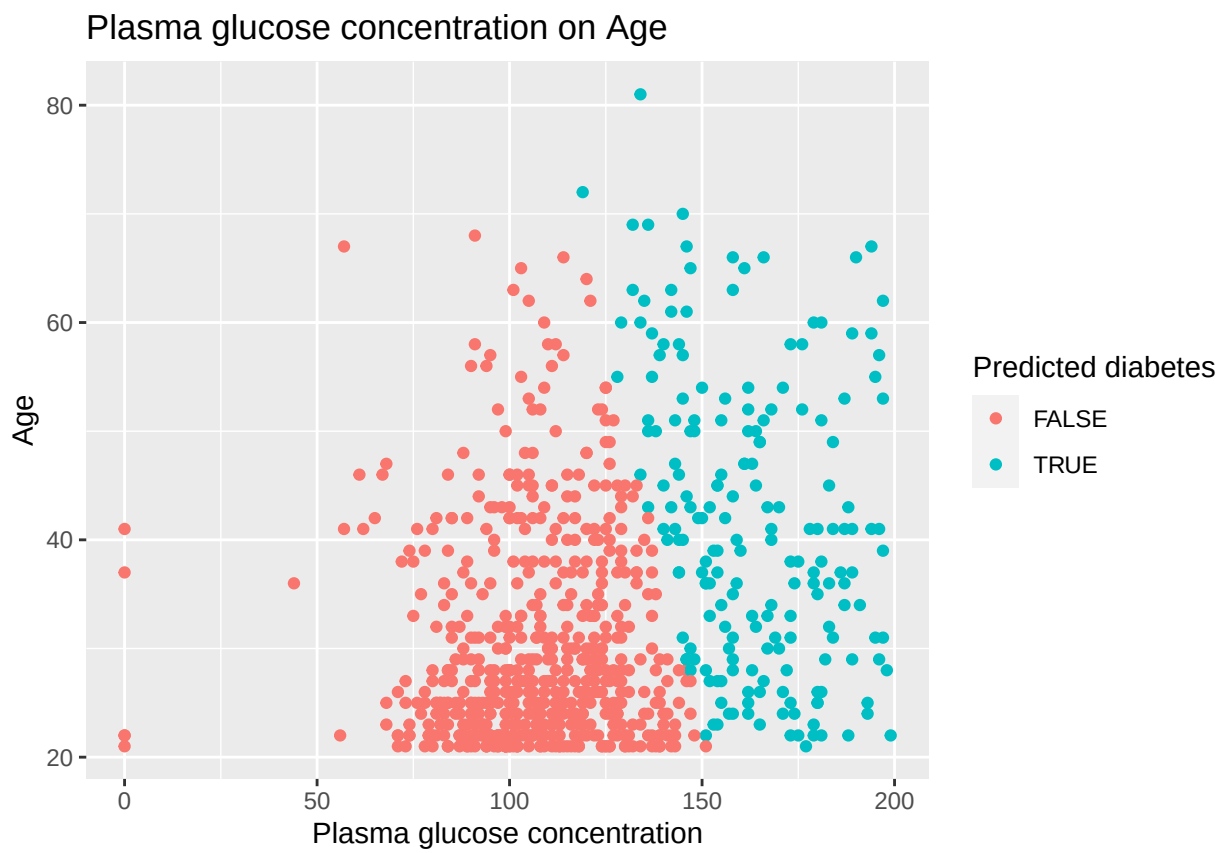


Yes, these two variables seem to be able to seperate the positive and the negative cases of diabetes along their axes. For example, the visible cluster of negative cases seem to have a low plasma glucose concentration and a low age in common. Also, along the x-axis (plasma glucose concentration) higher values seem to correspond do more positive cases of diabetes. These two features looks promising in their ability to separate (and predict) diabetes cases. Although, there are some observations which do not fit this pattern, so the classification rate will most likely not be perfect using only these two variables.

3.2

Train a logistic regression model with y=Diabetes as target x1=Plasma glucose concentration and x2 =Age as features and make a prediction for all observations by using r= 0.5 as the classification threshold. Report the probabilistic equation of the estimated model (i.e., how the target depends on the features and the estimated model parameters probabilistically). Compute also the training missclassification error and make a scatter plot of the same kind as in step 1 but showing the predicted values of Diabetes as a color instead.Comment on the quality of the classification by using these results.

```
##
## Call:
## glm(formula = diabetes ~ Age + Glucose_concentration, family = "binomial",
##     data = diabetes)
##
## Coefficients:
##                        Estimate Std. Error z value Pr(>|z|)
## (Intercept)           -5.912449   0.462620  -12.78  < 2e-16 ***
## Age                    0.024778   0.007374    3.36 0.000778 ***
## Glucose_concentration  0.035644   0.003290   10.83  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 993.48  on 767  degrees of freedom
## Residual deviance: 797.36  on 765  degrees of freedom
## AIC: 803.36
##
## Number of Fisher Scoring iterations: 4
```




```
## [1] 0.2630208
```

The probabilistic equation of the logistic regression model is given by:

$$p(x) = \frac{1}{1 + e^{-(b_0 + b_1 x_1 + b_2 x_2)}}$$

Where: - β_0 is the intercept of the model. - β_1 is the coefficient for Age. - β_2 is the coefficient for Glucose Concentration. - e is the base of the natural logarithm - x_1 and x_2 are the age and the glucose concentration.

The coefficients β_0, β_1, and β_2 are estimated from the data. Below is the summary of the model which includes these coefficients:

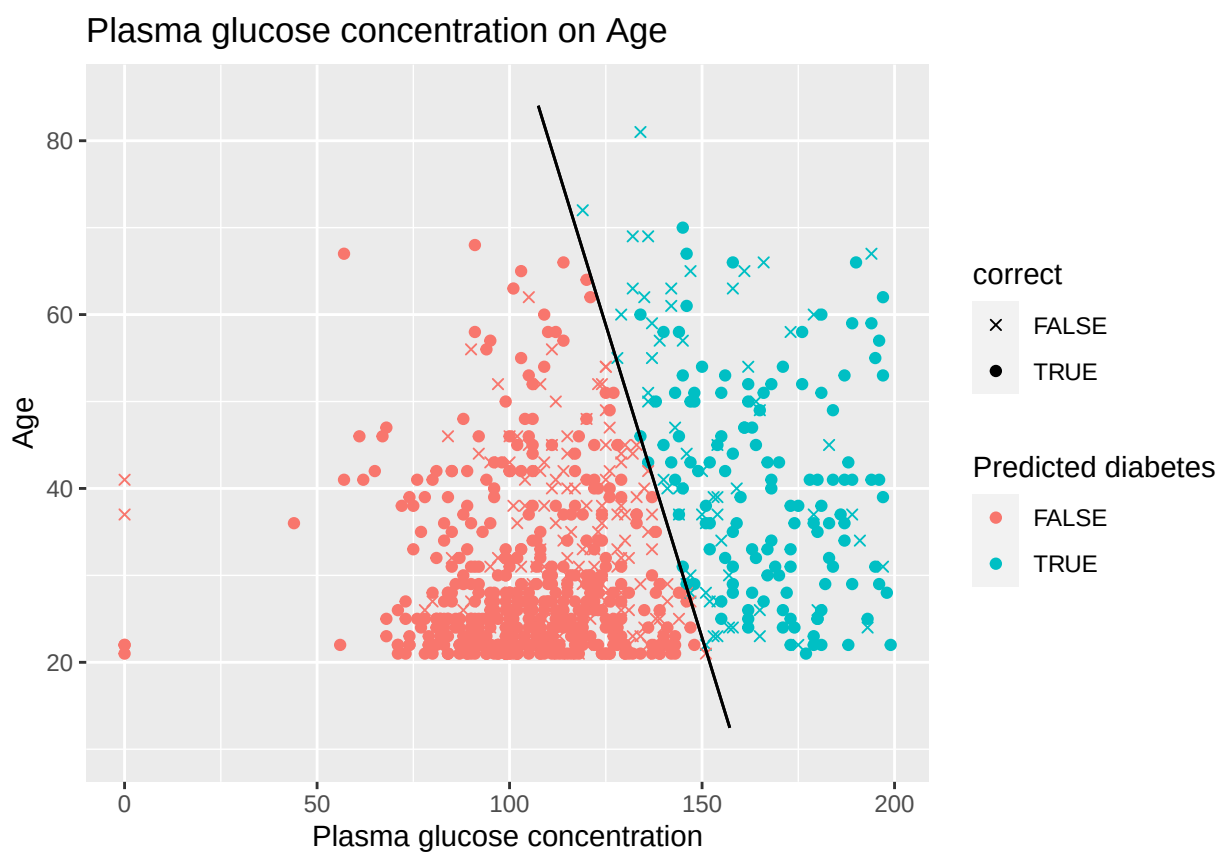
```
##
## Call:
## glm(formula = diabetes ~ Age + Glucose_concentration, family = "binomial",
##     data = diabetes)
##
## Coefficients:
##                        Estimate Std. Error z value Pr(>|z|)
## (Intercept)           -5.912449   0.462620  -12.78  < 2e-16 ***
## Age                    0.024778   0.007374    3.36 0.000778 ***
## Glucose_concentration  0.035644   0.003290   10.83  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 993.48  on 767  degrees of freedom
## Residual deviance: 797.36  on 765  degrees of freedom
## AIC: 803.36
##
## Number of Fisher Scoring iterations: 4
```

The missclassification rate is found 26.3%. That means that 26.3% cases were missclassified. The majority of the cases were correctly classified. This shows that the model works but it could be improved.

3.3

Use the model estimated in step 2 to a) report the equation of the decision boundary between the two classes b) add a curve showing this boundary to the scatter plot in step 2. Comment whether the decision boundary seems to catch the data distribution well.



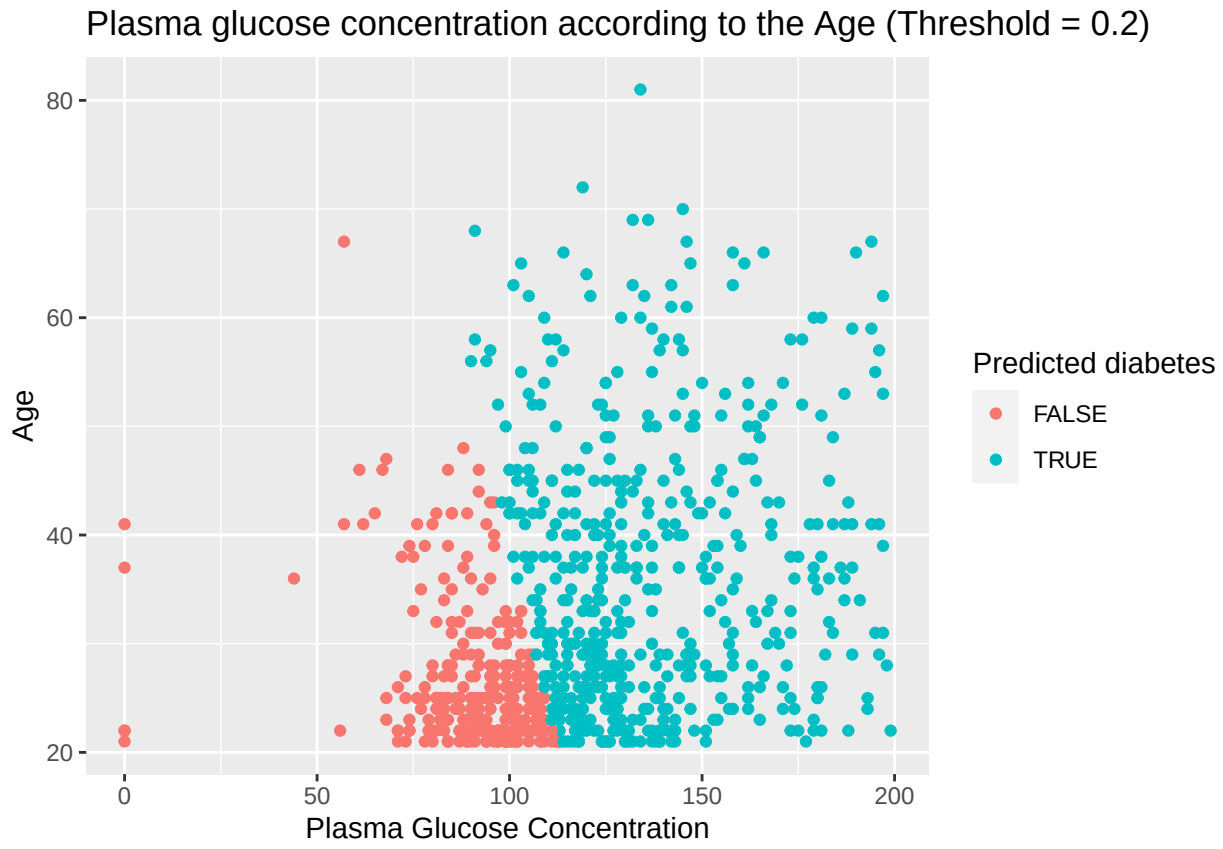
Probabilistic model is $\frac{1}{1+exp^{-(-5.91)+0.04*Glucose_{concentration}+0.02*Age}}$

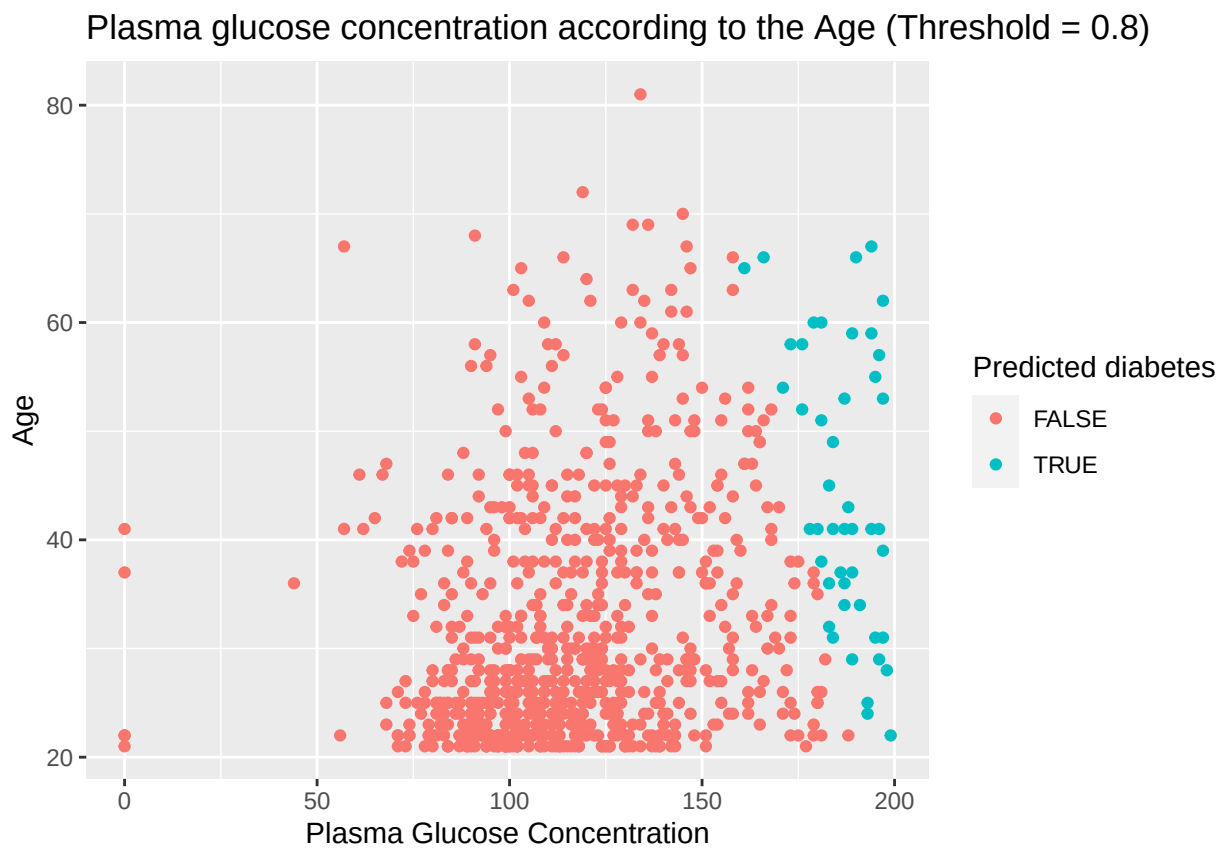
The curve is $x_2 = -\frac{\beta_0}{\beta_2} - \frac{\beta_1}{\beta_2}x_1$ $x_2 = -\frac{5.91}{0.02} - \frac{0.04}{0.02}x_1$

The curve seems to seperate well the predicted values. However, there are lots of incorrectly predicted values in both sides. That means that the model could be improved. This could be done either by adding more variables or by doing some variable transformations.

3.4

Make same kind of plots as in step 2 but use thresholds r= 0.2 and r= 0.8. By using these plots, comment on what happens with the prediction when r value changes.



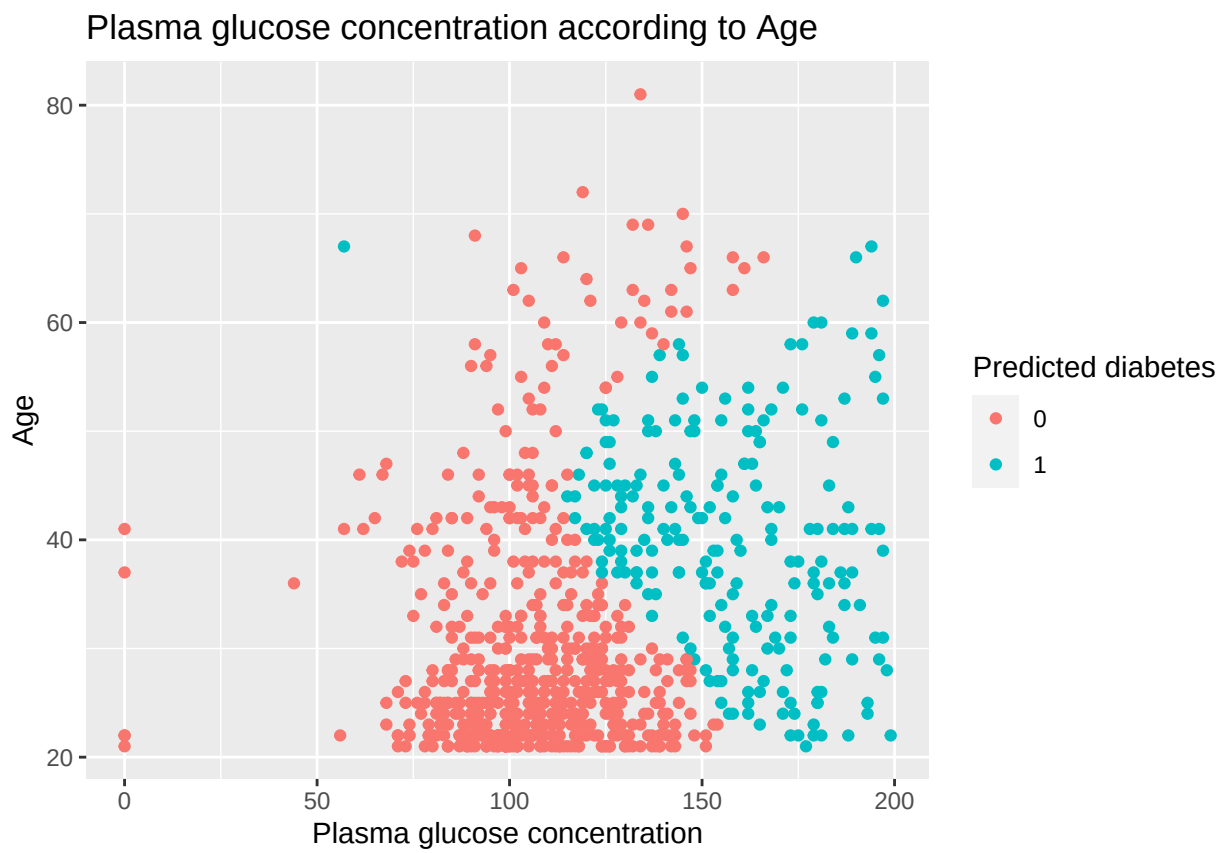


When the r value increases, the algorithm predicts decreased number of cases with diabetes. The opposite happens when r value decreases.

3.5

Perform a basis function expansion trick by computing new features z1=x1^4, z2=x1^3*x2, z3=x1^2*x2^2, z4=x1*x2^3, z5=x2^4 adding them to the data set and then computing a logistic regression model with y as target and x1,x2,z1,. . . ,z5 as features. Create a scatter plot of the same kind as in step 2 for this model and compute the training missclassification rate. What can you say about the quality of this model compared to the previous logistic regression model? How have the basis expansion trick affected the shape of the decision boundary and the prediction accuracy?

```
## Misclassification Rate: 0.2447917
```




```
## Misclassification Rate: 0.2447917
```

The boundary line is different in this model. The diabetes cases can not be seperated by a straight line from the ones without diabetes. The misclassification rate is 24.5%, which is better than in the previous logistic regression model. The basis expansion trick has made the shape of the decision boundary triangular and not linear. The prediction accuracy will now be 100%-24.5% which is equal to 75.5%.

Appendix

Chunk Label: setup

```r
knitr::opts_chunk$set(echo = TRUE)
knitr::opts_chunk$set(message = FALSE, warning = FALSE)

library(kknn)

library(ggplot2)
```

Chunk Label: 1_1_import

```r
# Reading data
data <- read.csv("~/Documents/Machine Learning/optdigits.csv", header=FALSE)

# The last columns identifies which digit (0-9) the data row corresponds to
colnames(data)[65] <- "digit"
data$digit <- as.factor(data$digit)

n <- dim(data)[1]
# 50% partition to train model
set.seed(12345)
id <- sample(1:n, floor(n*0.5))
train <- data[id,]
id1 <- setdiff(1:n, id)
# 25% partition to validate hyperparameter
set.seed(12345)
id2=sample(id1, floor(n*0.25))
valid=data[id2,]
# 25% partition to test the model
id3=setdiff(id1,id2)
test=data[id3,]
```

Chunk Label: 1_2_confusion_missclass

```r
#Makes a model which result is tested on the same data it is trained on
mod_traintrain <- kknn(formula =  digit ~. ,
                       train = train,
                       test = train,
                       kernel = "rectangular",
                       k = 30)

#Makes a model which result is tested on test data
mod_traintest <- kknn(formula =  digit ~. ,
                      train = train,
                      test = test,
                      kernel = "rectangular",
                      k = 30)

print("Confusion matrix for training data:")
table(train$digit, mod_traintrain$fitted.values, dnn = list("true","pred"))

print("Confusion matrix for test data:")
table(test$digit, mod_traintest$fitted.values, dnn = list("true","pred"))

print("Misclassification Error for train data:")
round(1-sum(diag(table(train$digit, mod_traintrain$fitted.values)))/nrow(train),3)
print("Misclassification Error for test data:")
round(1-sum(diag(table(test$digit, mod_traintest$fitted.values)))/nrow(test),3)
```

Chunk Label: 1_3_plot_eights

```
#Gets the probability matrix for each observation and digit from the model tested on train data
eights <- data.frame(mod_traintrain$prob)
#Copies the row.names from the train partition as they were shuffled
row.names(eights) <- row.names(train)

#Just keeps the digit probabilities for every observation representing the true eights (8)
eights <- eights[train$digit == "8",]
#Orders the data in decreasing order of the column representing the probability of digit eight (8)
eights <- eights[order(eights$X8,decreasing = T),]
#Extracts the two observations which the model were most sure of were eights (8).
output <- eights[1:2,]


heatmap(t(matrix(unlist(train[row.names(output)[1],-65])/16, ncol = 8)), Colv = NA, Rowv = NA)
heatmap(t(matrix(unlist(train[row.names(output)[2],-65])/16, ncol = 8)), Colv = NA, Rowv = NA)
```

Chunk Label: 1_3_plot_eights2

```
#Changes order to increasing
eights <- eights[order(eights$X8,decreasing = F),]

output <- eights[1:3,]


heatmap(t(matrix(unlist(train[row.names(output)[1],-65])/16, ncol = 8)), Colv = NA, Rowv = NA)
heatmap(t(matrix(unlist(train[row.names(output)[2],-65])/16, ncol = 8)), Colv = NA, Rowv = NA)
heatmap(t(matrix(unlist(train[row.names(output)[3],-65])/16, ncol = 8)), Colv = NA, Rowv = NA)
```

Chunk Label: 1_4_plot_k

```
# Creates a function which returns the missclassification
#errors on train-and validation data
# ... given argument k

my_knn <- function(k){

  # Makes a model which result is tested on the same data it is trained on
  mod_traintrain <- kknn(formula =  digit ~. ,
                         train = train,
                         test = train,
                         kernel = "rectangular",
                         k = k)

  # Makes a model which result is tested on the validation data
  mod_trainvalid <- kknn(formula =  digit ~. ,
                         train = train,
```

```
                        test = valid,
                        kernel = "rectangular",
                        k = k)




  return(data.frame(train = 1-sum(diag(table(train$digit,
                                             mod_traintrain$fitted.values)))
                    /nrow(train),
                    valid = 1-sum(diag(table(valid$digit,
                                             mod_trainvalid$fitted.values)))
                    /nrow(valid)))

}

# Applies the function from k = 1 to k = 30
knn_res <- lapply(1:30, my_knn)
# Rowbinds the list into a data.frame
knn_res <- do.call(rbind, knn_res)
# Adds a column for k
knn_res$k <- 1:30

# Plot with train fit as x and validation fit as y, using color aesthetic to indicate k

ggplot(data = knn_res, aes(x = train, y = valid, color = k)) +
  geom_point() +
  geom_text(label = knn_res$k, nudge_y =0.001)
```

Chunk Label: 1_4_comparison

```
# Testing the model using k = 3 on test data
mod_traintest <- kknn(formula =  digit ~. ,train = train, test = test,
                      kernel = "rectangular", k = 3)


# Comparing missclassing-error
round(data.frame(knn_res[3,1:2],
           test = 1-sum(diag(table(test$digit, mod_traintest$fitted.values)))
           /nrow(test)),
      3)
```

Chunk Label: 1_5_crossentropy

```
# Making a function to calculate cross-entropy loss for categorical data
# Argument y = the output of a kknn()
# Argument cl = a vector of all class labels (digits)

xentrop <- function(y, cl){
```

```r
  output <- c()

  for(i in 1:nrow(y$prob)){
    temp <- c()
    for(c in cl){
      temp[length(temp)+1] <- (y$fitted.values[i] == c)*log(y$prob[i,as.character(c)]+1e-15)
      output[length(output)+1 ] <- sum(temp)
    }
  }
  return(-sum(output))
}

# Creates a function which returns the cross-entropy-errors on train- and validation data
# ... given argument k

my_knn2 <- function(k){

  mod_traintrain <- kknn(formula =  digit ~. ,
                         train = train,
                         test = train,
                         kernel = "rectangular",
                         k = k)

  mod_trainvalid <- kknn(formula =  digit ~. ,
                         train = train,
                         test = valid,
                         kernel = "rectangular",
                         k = k)




  return(data.frame(train = xentrop(mod_traintrain, cl = unique(train$digit)),
                    valid = xentrop(mod_trainvalid, cl = unique(valid$digit))))




}

# Applies the function from k = 1 to k = 30
knn_res2 <- lapply(1:30, my_knn2)
# Rowbinds the list into a data.frame
knn_res2 <- do.call(rbind, knn_res2)
# Adds a column for k
knn_res2$k <- 1:30



# Plot with k as x and validation fit as y, using color aesthetic to indicate k
ggplot(data = knn_res2, aes(x = k, y = valid)) + geom_point()
```

Chunk Label: Question2.1

```
library(caret)
library(lattice)

parkinsons_model_data <- read.csv("parkinsons.csv")

df2 <-
  parkinsons_model_data[, c(
    "motor_UPDRS",
    "Jitter...",
    "Jitter.Abs.",
    "Jitter.RAP",
    "Jitter.PPQ5",
    "Jitter.DDP",
    "Shimmer",
    "Shimmer.dB.",
    "Shimmer.APQ3",
    "Shimmer.APQ5",
    "Shimmer.APQ11",
    "Shimmer.DDA",
    "NHR",
    "HNR",
    "RPDE",
    "DFA",
    "PPE"
  )]

set.seed(12345)
n <- nrow(df2) #number of rows in the dataset

id <- sample(1:n, floor(n * 0.6)) #shuffle and divide
train <- df2[id,] #60 train data
test <- df2[-id,] #40 test data

scaler <- preProcess(train)
train_data_scaled <- predict(scaler, train) #scaling the data
test_data_scaled <- predict(scaler, test)

model <- lm(motor_UPDRS ~ .-1, data = train_data_scaled)
```

Chunk Label: Question2.2

```
library(caret)
library(ggplot2)
library(lattice)

parkinsons_model_data <- read.csv("parkinsons.csv")

df2 <-
  parkinsons_model_data[, c(
```

```r
    "motor_UPDRS",
    "Jitter...",
    "Jitter.Abs.",
    "Jitter.RAP",
    "Jitter.PPQ5",
    "Jitter.DDP",
    "Shimmer",
    "Shimmer.dB.",
    "Shimmer.APQ3",
    "Shimmer.APQ5",
    "Shimmer.APQ11",
    "Shimmer.DDA",
    "NHR",
    "HNR",
    "RPDE",
    "DFA",
    "PPE"
  )]

set.seed(12345)
n <- nrow(df2)

id <- sample(1:n, floor(n * 0.6))
train <- df2[id,] #60 train data
test <- df2[-id,] #40 test data


scaler <- preProcess(train)
train_data_scaled <- predict(scaler, train)
test_data_scaled <- predict(scaler, test)

model <- lm(motor_UPDRS ~ .-1, data = train_data_scaled)

predict_columns<-setdiff(names(train_data_scaled),"motor_UPDRS")
train_x<-train_data_scaled[,predict_columns]
test_x<-test_data_scaled[,predict_columns]

train_predictions <- predict(model, newdata = train_x)
test_predictions <- predict(model, newdata = test_x)

# MSE calculation MSE = (1/n) * SUM(yi - y_hat_i)^2
train_mse <- mean((train_data_scaled$motor_UPDRS - train_predictions) ^ 2)
test_mse <- mean((test_data_scaled$motor_UPDRS - test_predictions) ^ 2)

cat("Train MSE:", train_mse, "\n")
cat("Test MSE:", test_mse, "\n")
summary(model)
```

Chunk Label: Question2.3 a

```r
Loglikelihood <-
  function(theta, sigma) {
```

```r
    #in the question theta is the vector of coefficients
    #in the formula xi is the independent variables,
    #beta is the coefficients of the regression model
    #yi is the observed values of the dependent variable
    #actual values of dependent variable. motor_UPDRS from scaled train data
    observations <- train_data_scaled$motor_UPDRS
    #predictors are the independent variables exclude motor_UPDRS
    predictors <- train_data_scaled[, !(names(train_data_scaled) %in% 'motor_UPDRS')]
    #matrix multiplication predictors and parameters of the model
    prediction <- as.matrix(predictors) %*% theta
    sum_square <- sum((observations - prediction)^2)
    n <- length(observations)
    log_likelihood <- (-n/2 * log(2 * pi * sigma^2) - 1/(2 * sigma^2) * sum_square)

    return(log_likelihood)

  }
```

Chunk Label: Question2.3 b

```r
Ridge<-function(theta,sigma,lambda){
  log_like_value<-Loglikelihood(theta, sigma)


  ridge_penalty <- lambda * sum(theta[-1]^2) #ridge penalty
  negative_ridge <- (-log_like_value) + ridge_penalty
  return(negative_ridge)
}
```

Chunk Label: Question2.3 c

```r
RidgeOpt <- function(lambda) {
  number_predictor<-ncol(train_data_scaled)-1
   #initial theta value,vector of zeros. start coeffs for the model
  initial_theta <- rep(0, number_predictor)
  initial_sigma <- 1 #initial sigma value set to one
  params <- c(initial_theta, initial_sigma)

  obj_func<-function(params){
    theta <- params[1:number_predictor] #extract theta
    sigma <- params[number_predictor + 1] #extract sigma
    Ridge(theta, sigma, lambda) #call ridge function

  }
  opt_res <- optim(par = params, fn = obj_func, method = "BFGS") #optim ridge
}
```

Chunk Label: Question2.3 d

```
DF<-function(lambda){

   global_train_data <- train_data_scaled #scaled train data
    #X design matrix all columns except one
    X <- as.matrix(global_train_data[, !(names(global_train_data) %in% 'motor_UPDRS')])
    #svd devide matrix to three other matrices. 'U', 'D','V' singular values are in D
    svd_results <- svd(X) #singular value decomposition
    d <- svd_results$d #extract single values
    degrees_freedom <- sum((d^2) / (d^2 + lambda))
    return(degrees_freedom)
}
  #lambda_value <- 0.1
  #df_lambda <- DF(lambda_value)
  #print(paste("degrees of freedom for lambda value 0.1:", df_lambda))
```

Chunk Label: Question 2.4

```
 opt_theta_1 <- RidgeOpt(1)$par
  opt_theta_2 <- RidgeOpt(100)$par
  opt_theta_3 <- RidgeOpt(1000)$par

  #cat("Optimal Theta for Lambda 1:\n", opt_theta_1, "\n\n")
  #cat("Optimal Theta for Lambda 100:\n", opt_theta_2, "\n\n")
  #cat("Optimal Theta for Lambda 1000:\n", opt_theta_3, "\n\n")

  predict_val <- function(theta, data) {
    theta_coefficients <- theta[1:(length(theta) - 1)]

    predictors <- as.matrix(data[, !(names(data) %in% 'motor_UPDRS')])

    if (ncol(predictors) != length(theta_coefficients)) {
      stop("Number of predictors and length of theta do not match.")
    }

    prediction <- predictors %*% theta_coefficients
    return(prediction)
  }


  train_pred_1 <- predict_val(opt_theta_1, train_data_scaled)
  test_pred_1 <- predict_val(opt_theta_1, test_data_scaled)

  train_pred_2 <- predict_val(opt_theta_2, train_data_scaled)
  test_pred_2 <- predict_val(opt_theta_2, test_data_scaled)

  train_pred_3 <- predict_val(opt_theta_3, train_data_scaled)
  test_pred_3 <- predict_val(opt_theta_3, test_data_scaled)
```

```r
  mse_train_1 <- mean((train_data_scaled$motor_UPDRS - train_pred_1) ^ 2)
  mse_test_1 <- mean((test_data_scaled$motor_UPDRS - test_pred_1) ^ 2)

  mse_train_2 <- mean((train_data_scaled$motor_UPDRS - train_pred_2) ^ 2)
  mse_test_2 <- mean((test_data_scaled$motor_UPDRS - test_pred_2) ^ 2)

  mse_train_3 <- mean((train_data_scaled$motor_UPDRS - train_pred_3) ^ 2)
  mse_test_3 <- mean((test_data_scaled$motor_UPDRS - test_pred_3) ^ 2)

cat("MSE for Lambda 1:\n", "Train MSE:", mse_train_1,"\n", "Test MSE:",mse_test_1, "\n\n")
cat("MSE for Lambda 100:\n", "Train MSE:", mse_train_2,"\n", "Test MSE:", mse_test_2, "\n\n")
cat("MSE for Lambda 1000:\n", "Train MSE:", mse_train_3,"\n", "Test MSE:", mse_test_3, "\n\n")

degrees_freedom_1<-DF(1)
degrees_freedom_2<-DF(100)
degrees_freedom_3<-DF(1000)


cat("Degrees of Freedom Lambda 1:", degrees_freedom_1, "\n")
cat("Degrees of Freedom Lambda 100:", degrees_freedom_2,"\n")
cat("Degrees of Freedom  Lambda 1000:", degrees_freedom_3,"\n")
```

Chunk Label: unnamed-chunk-1

```r
library(ggplot2)
library(dplyr)
library(readr)
```

Chunk Label: 3_1

```r
diabetes <- read.csv("~/Documents/Machine Learning/pima-indians-diabetes.csv"
                     , header=FALSE)
colnames(diabetes) <- c(     #renaming the columns for clarity
  "Amount_Pregnancies",
  "Glucose_concentration",
  "Blood_pressure",
  "Skin_thickness",
  "Insulin",
  "BMI",
  "diabetes_pedigree",
  "Age",
  "diabetes"
)
plot_diabetes <- ggplot(diabetes, aes(x=Glucose_concentration,y=Age,
                                      color = as.factor(ifelse(diabetes == 1, "True", "False"))))+
  geom_point() +
  labs(color = "diabetes")+
  xlab("Glucose concentration")+
  ylab("Age")+
```

```r
  ggtitle("Plasma glucose concentration according to Age")
print(plot_diabetes)
```

Chunk Label: 3_2

```r
logistic_model <- glm(diabetes ~ Age + Glucose_concentration, data = diabetes,
                      family = "binomial")
summary(logistic_model)

missclassification<-function(x){
  total_instances<-sum(x)
  accuracy <- sum(diag(x))/total_instances
  return(1-accuracy)
}

 #predicting probabilities using the logistic model
predicted_probabilities=predict(logistic_model, type="response")

#classifying based on the predicted probabilities with a threshold of 0.5
predicted_classification=ifelse(predicted_probabilities>0.5, "TRUE", "FALSE")

#creating a confusion matrix to compare actual and predicted classifications
confusion_matrix_train = table(diabetes$diabetes, predicted_classification)

#calculating misclassification rate using the confusion matrix
missclassification_rate<-missclassification(confusion_matrix_train)

diabetes$Predicted = predicted_classification

plot2 <- ggplot(diabetes, aes(x=Glucose_concentration,y=Age,color=Predicted))+
  geom_point() +
  labs(color = "Predicted diabetes")+
  xlab("Plasma glucose concentration")+
  ylab("Age")+
  ggtitle("Plasma glucose concentration on Age")
plot2

print(missclassification_rate)
```

Chunk Label: 3_2_2

```r
# Fitting the logistic regression model
logistic_model <- glm(diabetes ~ Age + Glucose_concentration, data = diabetes, family = "binomial")

# Displaying the summary of the model
summary(logistic_model)
```

Chunk Label: 3_3

```
diabetes$Pred<-0  #created one more column to the data set
diabetes[predict(logistic_model,newdata = diabetes,type="response")>0.5, "Pred"]<-1
diabetes$correct <- diabetes$Pred == diabetes$diabetes

plot3 <- ggplot(diabetes, aes(x=Glucose_concentration,y=Age,
                              color=as.logical(predicted_classification), shape = correct))+
  geom_point() +
  stat_function(fun=function(x) (-logistic_model$coefficients[1]/
                                   logistic_model$coefficients[2]-logistic_model$coefficients[3]
                                 /logistic_model$coefficients[2]*x),colour="black")+
  labs(color = "Predicted diabetes")+
  ylab("Age")+
  xlab("Plasma glucose concentration")+
  ggtitle("Plasma glucose concentration on Age")+
  ylim(c(10,85)) +
  scale_shape_manual(values = c(4, 19))
print(plot3)
```

Chunk Label: 3_4

```
#predictions for threshold 0.2
Pred_0.2=ifelse(predicted_probabilities>0.2, "TRUE", "FALSE")
diabetes["Predicted_0.2"] = Pred_0.2
plot_0.2 <- ggplot(diabetes, aes(x = Glucose_concentration, y = Age, color = Predicted_0.2)) +
  geom_point() +
  labs(color = "Predicted diabetes") +
  xlab("Plasma Glucose Concentration") +
  ylab("Age") +
  ggtitle("Plasma glucose concentration according to the Age (Threshold = 0.2)")
print(plot_0.2)

#predictions for threshold 0.8
Pred_0.8 = ifelse(predicted_probabilities > 0.8, "TRUE", "FALSE")
diabetes$Predicted_0.8 = Pred_0.8
plot_0.8 <- ggplot(diabetes, aes(x = Glucose_concentration, y = Age, color = Predicted_0.8)) +
  geom_point() +
  labs(color = "Predicted diabetes") +
  xlab("Plasma Glucose Concentration") +
  ylab("Age") +
  ggtitle("Plasma glucose concentration according to the Age (Threshold = 0.8)")
print(plot_0.8)
```

Chunk Label: unnamed-chunk-2

```
#create new variables Z1, Z2, Z3, Z4, and Z5
diabetes$Z1 <- diabetes$Glucose_concentration^4
diabetes$Z2 <- (diabetes$Glucose_concentration^3) * diabetes$Age
```

```r
diabetes$Z3 <- (diabetes$Glucose_concentration^2) * (diabetes$Age^2)
diabetes$Z4 <- diabetes$Glucose_concentration * (diabetes$Age^3)
diabetes$Z5 <- diabetes$Age^4

logistic_model2 <- glm(diabetes ~ Glucose_concentration + Age + Z1 + Z2 + Z3 + Z4 + Z5,
                       data = diabetes, family = "binomial")

# Predict probabilities and assign predictions
Predicted_prob2 <- predict(logistic_model2 , type = "response")
Predicted_classification2 <- ifelse(Predicted_prob2 > 0.5, 1 , 0)
confusion_matrix2 <- table(diabetes$diabetes, Predicted_classification2)
missclassification_rate <- mean(diabetes$diabetes != Predicted_classification2)
#adding Predicted2 column to the diabetes dataset
diabetes$Predicted2 <-Predicted_classification2
cat("Misclassification Rate:", missclassification_rate, "\n")
plot5 <- ggplot(diabetes, aes(x=Glucose_concentration, y=Age, color = as.factor(Predicted2))) +
  geom_point() +
  labs(color = "Predicted diabetes") +
  xlab("Plasma glucose concentration") +
  ylab("Age") +
  ggtitle("Plasma glucose concentration according to Age")
print(plot5)

cat("Misclassification Rate:", missclassification_rate, "\n")
```